iPRA with uniform clause use

Four proof instructions and one fixed equality clause

Research prepared for Vladimir Reshetnikov with ChatGPT

11 November 2026

Abstract

We give a smaller explicit presentation of full first-order intuitionistic primitive recursive arithmetic. The numerical terms remain raw scalar iterator terms; the public formulas are equations and one two-variable binder. A context stores reusable clauses with zero or one premise. One fixed clause, $\forall y\,(y = y)$, supplies reflexivity through the same use instruction as local assumptions and conditional closures. The resulting checker has four instruction families and a two-case computational equality operation. The reflexivity axiom and both evaluation cases are counted.

An explicit equivalence with the existing raw clause calculus preserves every public formula and numerical term. Composition with the repository's native/iPRA bridge therefore gives bi-interpretability on the entire numerical domain, with actual equality and identity graph $x = y$ in both composites. In a matched, fully validated JSON implementation, the complete normalized checker decreases from 3,046 to 2,912 bytes, from 853 to 807 significant tokens, and from 993 to 931 AST nodes. These are improvements within this explicit implementation family, not a global size record. The repository also contains substantially smaller compact encodings. The new result is a modest presentation simplification, not a claimed arithmetic breakthrough or an absolute minimum.

1 Result, comparison target, and scope

The comparison is pinned to ProveIt commit `5d85023b0688e03b75f0ec3a04bb43ef0bb51705`, under `Logic/PrimitiveRecursiveArithmetic/ConciseIPRA` [1]. At that snapshot the project already includes raw computational equality, clause contexts that remove a primitive Q connective, zero and full normalization, and several compact certificate encodings. None of those ideas is presented here as new.

The closest implementation is `Research/raw_clausal/code/clausal_eval.py`. Its five instruction families are Refl, Eval, clause Use, paired elimination, and paired introduction [2]. Its Eval operation has three cases. The repository's `NativeEval.lean` separately establishes that its equality-composition case is derivable from the other two [3]. We combine that established deletion with one further interface change: unconditional clauses and conditional clauses use the same premise-list mechanism. Reflexivity is the single initial unconditional clause, and a local assumption becomes an unconditional constant clause.

The main distinction is between a mathematical theory and its proof presentation. This construction changes the presentation. It retains the existing arithmetic interpretation, and proves exact equivalence before appealing to that interpretation. It neither identifies mere standard-model agreement with bi-interpretability nor replaces a proof by finite testing.

Feature	Existing raw clause calculus	This presentation
Numerical constructors	$x, 0, S, I$	$x, 0, S, I$
Formula constructors	$=, D$	$=, D$
Instruction tags	5	4
Initial fixed clauses	0	1 (reflexivity)
Cases in Eval	3	2
Local assumption premise	proof of $0 = 0$	empty list
Substitution	ordinary	ordinary
Numerical normalization	none	none

"Four" counts instruction families, not an encoding-independent number of logical principles. Eval has two specified cases; introduction has a specified mode test; clause use accepts zero or one premise. All this machinery, together with parsing, scope validation, JSON input, and the verdict driver, is included in the measured module. Splitting Eval into two separate instructions would give five instruction families, still together with the one fixed initial reflexivity clause.

2 Complete definition of the presentation

2.1 Terms and formulas

There is one numerical sort. Terms are finite expressions

$$t ::= x \mid 0 \mid S(t) \mid I(n, a; u.h).$$

Only the state variable u in the update body h is bound. In particular, u does not bind occurrences in n or a. An update is a fixed finite term template, possibly with passive numerical parameters. It is not a function variable, and there is no internal universal evaluator.

Formulas are

$$A ::= (s = t) \mid \mathsf{D}_{x,y} A.$$

The variables x, y are distinct and both are bound in the matrix. Bound names may be renamed before substitution. All substitutions in this report are capture avoiding. Terms remain raw: $I(0, a; u.h)$ and a are different syntax, even though their equality will be derivable. Write $\equiv_{\mathrm{syn}}$ for literal syntactic identity modulo bound-variable representation.

The ordinary reading, defined recursively on formulas, is

$$\begin{aligned} (s = t)^\circ &= (s = t), \\ (\mathsf{D}_{x,y} B)^\circ &= \exists x \big(B^\circ(x, 0) \to \forall y\, B^\circ(x, y) \big). \end{aligned}$$

The designated contradiction is the formula $0 = S0$.

2.2 Reusable clauses and the one fixed axiom

A context is a finite list of reusable clause occurrences

$$\langle \Pi; y.B(y) \rangle,$$

where Π is either $[]$ or $[A]$, and $y \notin \mathrm{FV}(\Pi)$. The binder y binds only the consequent. The ordinary readings are respectively

$$\langle []; y.B(y) \rangle^\circ = \forall y\, B^\circ(y), \qquad \langle [A]; y.B(y) \rangle^\circ = A^\circ \to \forall y\, B^\circ(y).$$

A clause is a context object, not a third formula constructor. Occurrences matter: two equal-looking clauses in different positions may have different proof origins.

Every proof has the fixed initial axiom clause

$$\boxed{\mathcal{R} = \langle []; y.(y = y)\rangle.} \tag{R}$$

For a fresh unused v, represent an ordinary assumption by

$$\mathsf{ass}(A) := \langle []; v.A\rangle.$$

This has the same ordinary meaning as A, since the numerical sort is inhabited by 0. No nonemptiness assumption is hidden in the notation.

2.3 The four instruction families

All displayed premises are in the retained context Γ unless a local extension is shown. Contexts retain the initial $\mathcal{R}$.

U: use a clause. For a selected occurrence $\langle [A_1, \ldots, A_k]; y.B(y)\rangle \in \Gamma$, where $k \in \{0, 1\}$, and any well-scoped term t,

$$\frac{\Gamma \vdash A_1 \quad \cdots \quad \Gamma \vdash A_k}{\Gamma \vdash B(t)}. \tag{U}$$

For $k = 0$ this instruction has no proof premises. In particular, use of $\mathcal{R}$ at t gives $t = t$, and use of $\mathsf{ass}(A)$ gives A.

V: computational equality. Define a partial operation on raw terms by

$$\Phi(s, t, I(0, s; u.h)) = t,$$
$$\Phi(I(c, a; u.h), t, I(Sc, a; u.h)) = h[t/u].$$

It is undefined in all other cases. Thus the required initializer or preceding iterator must match the source literally. The counter in the second line must literally have outer constructor S. The instruction is

$$\frac{\Gamma \vdash s = t \qquad \Gamma \vdash b = r}{\Gamma \vdash r = \Phi(s, t, b)} \quad \text{if } \Phi(s, t, b) \text{ is defined.} \tag{V}$$

This is not general term normalization. The two cases implement precisely one zero-iteration or successor-iteration equality step.

E: paired elimination.

$$\frac{\Gamma \vdash \mathsf{D}_{x,y}B(x, y) \qquad \Gamma, \langle [B(b, 0)]; y.B(b, y)\rangle \vdash A}{\Gamma \vdash A}. \tag{E}$$

The new numerical parameter b is fresh for Γ, A, and the matrix's passive parameters. The added clause is local to the branch.

I: paired introduction. Choose a term t and a fresh numerical parameter z. Put

$$\eta(B,t,z) = \begin{cases} z, & B \text{ is literally an equation and } t \equiv_{\mathrm{syn}} 0, \\ 0, & \text{otherwise.} \end{cases}$$

Then

$$\frac{\Gamma, \mathsf{ass}(B(t,\eta(B,t,z))) \vdash B(t,Sz)}{\Gamma \vdash \mathsf{D}_{x,y}B(x,y)}. \tag{I}$$

Here z is fresh for Γ, t, and the matrix's passive parameters. The binder used to express the constant clause $\mathsf{ass}(B(t,\eta))$ is fresh and unused, and does not bind z.

There are no other primitive logical, equality, computation, successor, or induction axioms or instructions. Weakening, fresh renaming, ordinary assumption cut, transport, and ordinary logical connectives below are admissible constructions, not additional trusted operations. The finite lists of clauses are reusable, so contraction is implicit in selecting the same occurrence repeatedly.

3 Equality and computation are recovered locally

Lemma 3.1 (The omitted evaluation case)**.** *From $s = t$ and $s = r$ one can derive $r = t$ using two zero-evaluation instructions and one instance of the initial reflexivity clause.*

Proof. Put $J = I(0, s; u.u)$. Clause use gives $J = J$. Zero evaluation applied to $s = r$ and $J = J$ gives $J = r$. Zero evaluation applied to $s = t$ and $J = r$ gives $r = t$. □

This is the exact equality-composition case present in the old raw clause checker. The deletion theorem is already formalized upstream as `EPrf.evalId` [3]; its application to the new clause interface does not require a new arithmetic theorem.

Corollary 3.2 (Equality algebra)**.** *Symmetry and transitivity of equality are admissible.*

Proof. For symmetry, combine $s = s$ and $s = t$ by Lemma 3.1. For transitivity, combine $t = u$ and the derived $t = s$ to obtain $s = u$. □

Lemma 3.3 (Both iterator equations)**.** *The following equations are derivable in every context:*

$$I(0, a; u.h) = a, \qquad I(Sn, a; u.h) = h[I(n, a; u.h)/u].$$

Proof. For the first, evaluate the reflexivities of a and $I(0, a; u.h)$. For the second, evaluate the reflexivities of $I(n, a; u.h)$ and $I(Sn, a; u.h)$. □

Lemma 3.4 (Congruence and one-template transport)**.** *For every fixed numerical template $h(u)$, from $s = t$ derive $h(s) = h(t)$; from $s = t$ and $h(s) = r$ derive $r = h(t)$.*

Proof. Let $J_0 = I(0, s; u.h)$ and $J_1 = I(S0, s; u.h)$. Zero evaluation gives $J_0 = s$ and $J_0 = t$. Successor evaluation of $J_0 = s$ with $J_1 = J_1$ gives $J_1 = h(s)$. Evaluate $J_0 = t$ with this last equality to obtain $h(s) = h(t)$. Apply Lemma 3.1 to that equation and $h(s) = r$ to obtain $r = h(t)$. □

The template may duplicate its argument and may contain nested iterator bodies. Its substitution is the ordinary capture-avoiding substitution already needed by the syntax. Two one-template transports give transport in an equation template $a(v) = b(v)$.

3.1 Recovering the case that the introduction test excludes

The syntax-sensitive introduction uses progress at an equation with the literal witness 0. Nevertheless it also derives every case introduction

$$\frac{\Gamma, \mathsf{ass}(B(t,0)) \vdash B(t,Sz)}{\Gamma \vdash \mathsf{D}_{x,y} B(x,y)}. \tag{Case}$$

Only the equation-root, literal-zero situation needs repair. Put $c = I(0,0;u.0)$. It is not literally 0, and the zero equation proves $c = 0$. Starting with $B(c,0)$, equation transport yields $B(0,0)$; insert the supplied branch proof to get $B(0,Sz)$, then transport back to $B(c,Sz)$. Instruction I at witness c uses case mode and has the original desired conclusion. Ordinary-assumption cut is justified directly by replacing uses of the selected unconditional constant clause by its supplied proof, with fresh parameters renamed first.

Transport in every Eq/D formula now follows by external induction on its syntax. For a D formula, eliminate the source at a fresh witness, introduce the target by Case at that witness, convert its zero assumption back to the source matrix, use the opened clause at Sz, and convert forward. Each recursive conversion concerns the strictly smaller matrix.

Substitution qualification. Arbitrary numerical substitution can change a nonliteral witness into literal 0, changing I's mode. We therefore do *not* claim that the same proof tags survive arbitrary substitution by a direct induction. The local proof translations below keep every witness literally unchanged. General substitution admissibility can subsequently be transferred through the established equivalence with the native calculus. This avoids a common circularity in arguments about this mode test.

4 Exact equivalence with the existing clause calculus

Let $\mathsf{CE5}$ be the existing raw clause calculus [2]. Its public language is exactly the language above, it has a primitive Refl instruction, and every clause has one antecedent. An ordinary assumption A is encoded as $\langle [0=0]; v.A \rangle$. Let $\mathsf{CE5}_2$ be the same calculus after deleting the identity case $\Phi(s,t,s) = t$ from Eval.

Proposition 4.1. $\mathsf{CE5}$ *and* $\mathsf{CE5}_2$ *derive exactly the same sequents.*

Proof. Every $\mathsf{CE5}_2$ rule is a $\mathsf{CE5}$ rule. In the other direction, recursively replace an identity-case Eval by Lemma 3.1, using the primitive reflexivity instruction available in $\mathsf{CE5}_2$. Copy all other nodes. The replacement preserves the end formula, all terms, and the retained context; no term substitution into proofs is involved. □

Theorem 4.2 (Literal preservation of public formulas)**.** *For every public Eq/D formula A,*

$$\vdash_{\mathsf{CE5}} A \quad \Longleftrightarrow \quad \vdash_{\mathsf{CE5}_2} A \quad \Longleftrightarrow \quad \mathcal{R} \vdash_{\mathsf{UC}} A.$$

The same equivalence holds with ordinary retained formula assumptions, represented by their corresponding constant clauses. The proof translations leave the numerical terms and public formulas literally unchanged.

Proof. For $\mathsf{CE5}_2 \to \mathsf{UC}$, add $\mathcal{R}$ at the beginning of the target context and maintain the correspondence between clause occurrences. A Refl node at t becomes U on $\mathcal{R}$ at t. Keep each conditional

clause introduced by E, and keep the translated proof of its antecedent when it is used. Replace each ordinary-assumption clause $\langle [0 = 0]; v.A \rangle$ by $\mathsf{ass}(A)$; its U node no longer requires the old proof of $0 = 0$. Copy V, E, and I with recursively translated subproofs. Every DI witness and every eigenparameter scope is unchanged. An induction on the proof tree establishes the same end formula at every node under the corresponding context.

Conversely, for $\mathsf{UC} \to \mathsf{CE5}_2$, remove the initial $\mathcal{R}$ and map a use of it at t to Refl at t. Replace a constant unconditional assumption clause by $\langle [0 = 0]; v.A \rangle$ and insert Refl at 0 as its U premise. Keep conditional clauses with their translated premise proofs. Again copy V, E, and I. Induction over the proof tree proves the exact context and end-formula invariant.

The public checker starts with only $\mathcal{R}$ and adds clauses only through E and I, so these cases exhaust its reachable context entries. For arbitrary externally stipulated unconditional clauses, use $\langle [0 = 0]; y.B(y) \rangle$ in the old calculus and supply Refl at zero when the clause is used; the same argument extends the context correspondence. □

If inference nodes alone are counted, $\mathsf{CE5}_2 \to \mathsf{UC}$ never increases their number. The reverse translation adds at most one Refl node per unconditional local-assumption use, hence at most doubles that count. Deleting the identity Eval case replaces its node by two Eval nodes and a reflexivity node. These bounds do not claim comparable bounds on serialized term labels, since a derived term can expand under substitution.

4.1 Connecting to the Lean native calculus

The native system EPrf has Eq/Q/D formulas. Its Q reading is $Q_y B = B(0) \to \forall y B(y)$. The existing raw-clause translation replaces it by the Eq/D macro

$$q_y B(y) := \mathsf{D}_{w,y} B(y), \qquad w \notin \mathrm{FV}(B). \tag{q}$$

To see its interface directly, Case introduces $q_y B$ from $B(0) \vdash B(Sz)$. E on $q_y B$ opens the clause $\langle [B(0)]; y.B(y) \rangle$, because the existential coordinate is unused. With a supplied base proof, U therefore specializes it at any term. Conversely, retaining this clause suffices to introduce $q_y B$.

These constructions translate the native QE and DE operations into clause operations. A native $Q_y B$ is interderivable with $D_{w,y} B$; structural induction lifts this equivalence through the surrounding binders. The reverse translation of a reachable new context is particularly short:

New clause origin	Native proof interface
$\mathcal{R}$	Refl proof provider
$\mathsf{ass}(A)$	ordinary hypothesis A
$\langle [B(b,0)]; y.B(b,y) \rangle$	hypothesis $Q_y B(b,y)$.

Thus U becomes Refl, Hyp, or QE; V, E, and I translate directly. This also gives a direct soundness induction into EPrf.

The upstream theorem `eprf_iff_nprf` identifies EPrf with the native kernel. The full raw-clause formula translation and its round trips are written out in the frozen reference fragment supplied with this package [2]. This report proves the new local equivalence; it does not claim a newly compiled Lean verification of the Python checker.

5 Ordinary logic and the iPRA bi-interpretation

5.1 An explicit logical interface

The small formula grammar does not restrict the theory to an equational or prenex fragment. All first-order intuitionistic connectives have effective macros. The following definitions make their syntax explicit; their native natural-deduction rules and ordinary meanings are proved in `NativeGate`, `NativeLogic`, and the native bridge [4, 6].

First put

$$C(v,a,b) := I(v,a;u.b), \qquad 1 := S0, \qquad p(v) := I(v,0;u.C(u,1,0)).$$

The bound u in C is unused. Computation gives $C(0,a,b) = a$ and $C(Sn,a,b) = b$; p is the parity iterator.

An equation/formula selector $L_v(s,t;B)$ is defined recursively by

$$\begin{aligned} L_v(s,t;a=b) &:= (C(v,s,a) = C(v,t,b)),\\ L_v(s,t;D_{x,y}B) &:= D_{x,y}L_v(C(y,0,s),C(y,0,t);B). \end{aligned}$$

Here each displayed binder is first chosen fresh for v,s,t. Set $G_v(B) := L_v(0,0;B)$. A general formula selector is

$$\begin{aligned} \mathrm{Sel}_v(s=t,B) &:= L_v(s,t;B),\\ \mathrm{Sel}_v(D_{x,y}A,B) &:= D_{x,y}\,\mathrm{Sel}_v(A,G_y(B)), \end{aligned}$$

with binders fresh for v,B. These are structural recursions on finite syntax, with no semantic test or quantification over formulas. The selector is equivalent to A at $v=0$ and to B at $v=Sn$; $G_0(B)$ is provable and $G_{Sn}(B)$ is equivalent to B.

Use the dummy-witness q from (q), and define

$$\begin{aligned} \bot^* &:= (0 = S0),\\ A \to^* B &:= q_y\,\mathrm{Sel}_y(A,B),\\ A \wedge^* B &:= q_y G_y(\mathrm{Sel}_{p(y)}(A,B)),\\ \exists^* x\,A(x) &:= D_{x,y}G_y(A(x)),\\ A \vee^* B &:= \exists^* v\ \mathrm{Sel}_v(A,B),\\ \forall^* x\,A(x) &:= A(0) \wedge^* q_y A(y). \end{aligned}$$

All auxiliaries are fresh. For example, the existential macro has ordinary meaning $\exists x A(x)$ because its zero antecedent is true and its successor values are $A(x)$. The conjunction macro can extract B at $y=1$ and A at $y=2$, while its zero instance is true. Numerical cases supply its reverse direction. No classical excluded-middle rule has been introduced by these definitions.

Equation induction follows directly: from $E(0)$ and a step $E(z) \vdash E(Sz)$, introduce $q_y E(y)$ using I with the unused witness 0, eliminate it, and use the resulting clause at the target. The progress mode remains restricted to literal equations. Extending it to all formulas would change the intended induction strength.

5.2 The ordinary scalar theory

To give the binder notation a conventional first-order realization, replace each fixed iterator body $h(u,\vec{a})$ by a staged function symbol $i_h(n,b,\vec{a})$, with its two recursion equations. Each description

uses previously available descriptions, and each is finite. Let T_I be ordinary intuitionistic first-order logic with equality, those equations, contextual induction for literal equations, and

$$\forall x\,(x = 0 \vee \exists y\, x = Sy), \qquad (0 = S0) \to \bot. \tag{Bridge}$$

The last sentence is required in an ordinary theory with genuine falsity: positive numerical equations alone also have a singleton model. It is the explicit falsity bridge for the native choice $\bot^* = 0 = S0$, not an unmentioned extra axiom of the four-instruction checker. Its native reading is derivable, as are the coverage and induction readings.

Under the ordinary readings, U is universal instantiation with zero or one implication premise; V is equality plus an iterator equation; E is existential elimination. In case mode I uses zero/successor coverage. In progress mode it uses equation induction while temporarily assuming the zero instance. The initial clause $\mathcal{R}$ is an equality theorem. Thus every new inference is sound in T_I. The recovered logical rules, cases, equation induction, and explosion give the converse presentation.

5.3 Arithmetic translation and its dependency order

The repository establishes the whole-domain scalar/iPRA bridge, including the following order of constructions [5, 6]:

1. Build a scalar predecessor P and prove $P0 = 0$ and $P(Sn) = n$, using equations, coverage, and equation induction.
2. Obtain truncated subtraction and a term reflecting actual equality. This is reflection for all domain elements, not merely standard numerals.
3. Translate each fixed quantifier-free formula into a characteristic equation. Equation induction then yields its quantifier-free induction.
4. Construct pairing and projections, and compile each fixed primitive recursive description into a scalar iterator term.

These are inherited results, not assumed extra primitives of UC. The reference directory includes the written predecessor, equality reflection, primitive recursion, and ordinary-bridge proofs. They are not part of the executable checker's trusted code.

For clarity, the recursion compiler has a concrete form. If

$$f(0, \vec{a}) = g(\vec{a}), \qquad f(Sn, \vec{a}) = h(n, f(n, \vec{a}), \vec{a}),$$

and π, L, H are the constructed pairing and projections, set

$$\begin{aligned} V(n, \vec{a}) &= I(n, \pi(0, g^*(\vec{a})); u.\pi(SL(u), h^*(L(u), H(u), \vec{a}))), \\ f^*(n, \vec{a}) &= H(V(n, \vec{a})). \end{aligned}$$

Equation induction proves $L(V(n, \vec{a})) = n$; the projection identities then give the required defining equations. The reverse translation into iPRA interprets a scalar iterator by primitive recursion whose update ignores its old-counter input.

Theorem 5.1 (Bi-interpretability, not only equiconsistency)**.** *The ordinary realization of* UC *is bi-interpretable with full first-order iPRA. Both interpretation domains are the entire numerical domain, equality is actual equality, and both numerical composite isomorphisms are defined by $x = y$. Both formula composites are provably equivalent to the original formulas. In particular, the presentations are equiconsistent.*

Proof. Theorem 4.2 is identity on the common numerical terms and public formulas. The raw-clause/native equivalence inserts only an unused existential coordinate for Q and proves its formula round trips. The upstream native/scalar/iPRA bridge provides faithful proof translations and both remaining round trips [6]. Compose them.

More explicitly, after translating a fixed function description in both directions, its double translation satisfies the original defining equations. Induction on the equality of values proves uniqueness of each fixed recursion, and external induction on the finite description proves that original and translated operations agree everywhere. The translations do not replace the domain by a cut, a code set, or a quotient. The identity graph is therefore internally total and bijective and preserves all operations. Structural induction on formulas, using the displayed ordinary meanings of D and the logical macros, proves their composites. The falsity bridge carries the designated contradiction in both directions. □

The particular upstream Lean statements are:

```
eprf_iff_nprf, native_iff_pra, pra_iff_native,
native_roundTrip, pra_native_roundTrip,
native_inconsistent_iff.
```

We inspected their source at the pinned commit. This run did not rebuild Lean and did not formalize the new checker in Lean or Rocq. The new local equivalence is a written proof with executable certificate translations and separate finite validation.

6 The complete checker and its certificate format

6.1 Scope representation

The standalone module `code/checker.py` uses integers and lists:

Syntax	JSON representation
Zero	`0`
Free numerical parameters	negative integer levels
Bound variables	positive de Bruijn indices starting at `1`
$S(t)$	`[0,t]`
$I(n, a; u.h)$	`[n,a,h]`
$s = t$	`[s,t]`
$D_{x,y}B$	`[B]` (inside B, y is 1 and x is 2)

At ambient scope k, allowed free levels are $-k, \ldots, -1$. The root goal has scope zero. A single recursive syntax validator checks term/formula sort, allowed constructors, and binder depth. A successor requires the literal tag zero. Exact Python integer typing excludes Boolean and floating-point aliases. The shared validator is the previous term and formula recursion factored by a sort flag; it does not enlarge the language.

Opening a binder replaces its matching positive index, decrements larger indices, and respects nested binders. Every replacement value is a term validated at binder depth zero, so it has no positive free indices and requires no additional lifting when inserted. Internal bound indices in the replacement remain internal. The iterator binds in its third child only; D binds two variables. Opening the outer x coordinate of a D matrix uses initial depth one, preserving its y coordinate.

6.2 Every proof instruction

Input is exactly an object with keys `goal` and `proof`. The goal is a closed Eq/D formula. Proof records are:

`["u", index, target, premises]`
: Select the clause at the nonnegative context index, instantiate its bound variable at `target`, and check the list of subproofs against its antecedent list. For an unconditional clause this list must be `[]`; for a conditional clause it must contain exactly one proof.

`["v", first, second]`
: Derive both premise equations and apply the specified two-case evaluation.

`["e", goal, major, branch]`
: Derive the paired major formula, allocate a fresh level, add its conditional clause, and check the branch against the stated outer goal.

`["i", goal, witness, branch]`
: Check the paired goal and witness in the old scope, allocate a fresh level, add the selected unconditional local assumption, and check the successor instance in the branch.

The initial context is `[([], [1,1])]`, the encoding of $\mathcal{R}$. Thus a complete proof of $0 = 0$ is

```
{"goal": [0, 0], "proof": ["u", 0, 0, []]}
```

There are no ignored proof payloads: even the zero-premise case checks that the supplied premise list derives the empty antecedent list. Every certificate-supplied field affects a checked syntax or proof condition.

6.3 Soundness invariant and resource convention

Simultaneously with recursion on the proof tree, maintain that the returned result is a well-scoped formula, all retained antecedents are well-scoped, and a retained body is well-scoped with one extra binder. The initial reflexivity clause satisfies this invariant. U preserves it by substitution; V by matching well-formed iterator terms; E and I by checked opening and lexical extension. A nonzero well-formed counter of list length two is a successor, so V's short counter check is justified by this invariant. Using a D formula as an equation fails destructuring. Iterators and variables cannot masquerade as successors.

E checks its conclusion before adding the fresh parameter. I similarly checks its witness and conclusion in the old scope. These ordering checks prevent eigenvariable escape. Negative clause indices are explicitly rejected; Boolean values cannot exploit Python's integer equality.

The driver prints `ACCEPT` and exits zero only on successful verification. Exceptions print `REJECT` and exit nonzero. The module uses no trusted `assert`, `eval`, solver, search procedure, or external theorem service. On a finite JSON tree the abstract algorithm terminates by structural recursion; ordinary host memory and recursion limits can reject extremely large otherwise valid proofs. Completeness of the formal calculus is understood with adequate resources, as for the comparison checkers.

7 Reproducible measurements and validation

7.1 Matched size comparison

Measurements use CPython 3.12.14. Remove genuine docstrings, normalize the whole module with `ast.unparse`, and retain the final newline. Count UTF-8 bytes, significant tokenizer tokens, and

all AST nodes. Comments and layout are normalized away; imports, validation, proof checking, JSON input, and the verdict driver remain. Gzip level nine is informational.

Complete module	Bytes	Tokens	AST	Gzip
Original raw clause checker	3046	853	993	990
Original, shared validator	3039	825	958	1008
Eval2, original validator	2999	848	986	979
Eval2, shared validator	2992	820	951	996
Uniform clauses, shared validator	**2912**	**807**	**931**	994

The total reduction relative to the original is 134 normalized bytes (4.40%), 46 significant tokens (5.39%), and 62 AST nodes (6.24%). With exactly the same shared validator, clause unification alone saves 80 bytes, 13 tokens, and 20 AST nodes. The redundant Eval-case deletion saves another 47 bytes, 5 tokens, and 7 AST nodes. Validator sharing saves 7 bytes, 28 tokens, and 35 AST nodes. The total gzip size increases by four bytes relative to the original, so there is no compressed-size victory.

This table is an ablation comparison, not a comparison against an artificial lower bound. In particular, the pinned repository has a 516-byte compact guarded codec and a 788-byte native Eq/D public-goal checker, with different syntax and validation contracts [1]. It also reports 256-token and 278-node minima in different programs. The new 2,912-byte module does *not* beat those global recorded coordinates. The delivered result is a strictly smaller, fully checked implementation of this explicit clause presentation. No claim that it is the absolutely simplest iPRA theory is made.

7.2 Proof translations and finite checks

The package includes an untrusted compiler replacing each old identity Eval by its two-zero-step derivation, and checked bidirectional translations between the Eval2 clause calculus and uniform clauses. Each compiler checks its source and target using the respective standalone kernels. They are evidence and tools for producing proofs, not dependencies of `code/checker.py`.

The portable validation suite exercises reflexivity, both computation cases, equality composition and transport, closure specialization, paired elimination, both introduction modes, used induction hypotheses, nested binders, and capture-avoiding witnesses. Negative tests target forged goals, out-of-scope parameters, loose indices, invalid successor tags, inappropriate progress, false evaluation matches, wrong premise lists, and malformed JSON data. It checks the source and translated proofs, their literal end formulas, and the reverse translations. Full-process checks run both ordinary Python and `python -O`. All 37 positive theorem certificates, 37 exact-goal translation round trips, 34 adversarial certificates, seven malformed input documents, and 56 inherited regression cases passed their expected outcomes. There were 484 full-process checks, including ordinary and optimized Python and readable and normalized kernels. The detailed receipt is `evidence/validation_summary.json`, generated by the shipped suite.

Finite validation can find implementation mistakes, but cannot prove soundness for every proof or bi-interpretability in all nonstandard models. Those claims rest on the written translations and the inherited arithmetic bridge. The separately supplied review records an independent assistant's audit of the final kernel; it is not external human peer review or a proof-assistant certificate.

8 Reproduction and delivered files

From the extracted archive directory, run:

```
python3 reproduce.py
python3 code/checker.py < examples/reflexivity.json
python3 build.py
```

The first command remeasures the complete modules and runs the portable proof-translation and rejection checks. The second demonstrates the standalone checker. The third rebuilds this report with PDFLaTeX, with shell escape disabled. The supplied `.tex` is standalone and can also be compiled directly in a directory containing no other source files. The executable artifacts need only Python's standard library; PDF rebuilding requires the TeX packages named in this source's preamble.

The main deliverables are this report, its editable source, the standalone checker and normalized source, proof translators and examples, the matched comparison modules, the portable validation suite, size and validation receipts, an independent review, and source provenance. Frozen reference material is attributed to the pinned ProveIt snapshot and distributed under its MIT No Attribution license. References are included for auditability; they are not imported by the trusted checker.

9 Further research questions

1. **Formalize the local presentation equivalence.** Add a small uniform-clause derivation type to the Lean project and prove its equivalence to `EPrf` or a clause intermediary. The context-origin invariant above should avoid rebuilding the arithmetic bridge.
2. **Combine explicit validation with the compact codec.** Can a similarly transparent, fully checked clause engine compete with the compact repository frontier? This report establishes a matched improvement only; scope and syntax costs must remain charged in any further comparison.
3. **Remove more arithmetic structure.** The raw-clausal report's successor-rigidity principle gives induction for zero equations, but a noncircular equality-reflection bootstrap remains open there. Likewise, the self-seeded iterator development still needs assumptions whose deletion requires a genuine new proof. Neither unresolved route is used here.
4. **Measure proof size as well as checker size.** The identity Eval deletion lengthens some equality certificates; removing neutral premises shortens local assumption uses. Quantify these effects on large arithmetic proofs rather than only small examples.
5. **Seek a principled lower bound.** Source length depends on the programming language, certificate grammar, validation contract, and runtime convention. Any meaningful optimality theorem must fix these first. The repository's achieved minima are upper bounds on an unknown optimum.

References

[1] VladimirReshetnikov/ProveIt, *Concise presentations of intuitionistic primitive recursive arithmetic*, pinned README and package index, commit `5d85023b0688e03b75f0ec3a04bb43ef0bb51705`. Repository snapshot.

[2] Same repository, `Research/raw_clausal/README.md`, `Research/raw_clausal/theory/equivalence.tex`, and `Research/raw_clausal/code/clausal_eval.py`. Raw-clausal package.

[3] Same repository, `Lean/ConciseIPRA/NativeEval.lean`, especially `EPrf.evalId`, `EPrf.trOne`, and `eprf_iff_nprf`; see also `Research/equality_from_iteration/`. Computational-equality Lean source.

[4] Same repository, `Lean/ConciseIPRA/NativeGate.lean` and `Lean/ConciseIPRA/NativeLogic.lean`; formula selectors, gates, and the natural-deduction rules for the native logical macros.

[5] Same repository, `Research/certified_zero/reference/`: `scalar_predecessor.tex`, `equality_reflection.tex`, `primitive_recursion.tex`, `inherited_choice_basis.tex`, and `inherited_ordinary_bridge.tex`.

[6] Same repository, `Lean/ConciseIPRA/BiInterpretation.lean` and `Lean/ConciseIPRA/NativeBiInterpretation.lean`; faithful interpretation, both formula round trips, and equiconsistency. Native/iPRA round-trip theorems.